

Understanding Hyperparameter Sensitivity: A Comprehensive Empirical Study Across Multiple Classification Algorithms and Diverse Datasets

Authors: Dhvani Balchandani & Nida S

Course: Machine Learning

Date: December 2025

Abstract

Machine learning practitioners often struggle with selecting optimal hyperparameters, yet little systematic research examines how sensitive different algorithms are to these choices across diverse datasets. This study addresses this gap by conducting an extensive empirical investigation of hyperparameter sensitivity across four classical machine learning algorithms (Decision Trees, k-Nearest Neighbors, Logistic Regression, and Support Vector Machines) applied to eight diverse medical and behavioral classification datasets.

Through 609 carefully designed experiments, we systematically varied key hyperparameters while holding others constant, measuring their impact on model performance. Our findings reveal that hyperparameter sensitivity is highly context-dependent: the distance metric choice in KNN produced a 30.5 percentage point AUC swing on SMS Spam data, while the split criterion in Decision Trees (Gini vs. Entropy) showed negligible impact across all datasets (average variance <0.000067). We identified that tree depth parameters, KNN's distance metric, and SVM's regularization strength are the most critical hyperparameters requiring careful tuning, while penalty type in Logistic Regression and split criterion in Decision Trees can safely use defaults.

Novel Contributions:

1. Comprehensive Sensitivity Hierarchy ranking 12 hyperparameters across 4 algorithms and 8 datasets, identifying which parameters require tuning versus which can use defaults
2. Dataset-Algorithm Interaction Analysis revealing how high-dimensional sparse data amplifies sensitivity patterns
3. Failure Configuration Catalog documenting specific parameter combinations causing catastrophic failures where models achieve $>85\%$ accuracy while completely failing on minority classes

Keywords: Hyperparameter Sensitivity, Parameter Tuning, Classification Algorithms, Medical Datasets, Model Optimization

1. Introduction

1.1 The Hyperparameter Tuning Challenge

Hyperparameter selection remains one of the most critical yet poorly understood aspects of machine learning practice. While substantial research addresses how to search the hyperparameter space efficiently (grid search, random search, Bayesian optimization), far less work examines which parameters actually need tuning and under what circumstances.

Consider a medical researcher building a diagnostic tool for heart disease with limited computational resources. They face questions like: Should I spend time tuning the regularization parameter in my SVM, or can I trust the default? Does the choice between Gini and Entropy splitting matter for my Decision Tree? Is the distance metric in KNN worth validating, or is it just an implementation detail?

Current practice often involves either expensive exhaustive grid searches (tuning everything) or blind faith in defaults (tuning nothing). A middle path requires understanding which hyperparameters are inherently sensitive versus robust, and how this sensitivity varies with dataset characteristics.

1.2 Research Gap

Most machine learning papers report final model performance ("our optimized SVM achieved 95% accuracy") without revealing the sensitivity landscape how much performance varies across different hyperparameter settings. This creates several problems:

- Unknown Cost of Not Tuning: Practitioners don't know whether using default parameters costs them 1% or 20% in performance
- Inefficient Search Strategies: Grid searches allocate equal computational budget to all parameters, even those with minimal impact
- Hidden Failure Modes: Some parameter combinations cause catastrophic failures that aren't obvious from final results

Existing hyperparameter studies focus on optimization algorithms or compare final model performance across datasets, but rarely provide actionable guidance on which specific parameters drive performance variance and which can be ignored.

1.3 Research Objectives

This study systematically quantifies hyperparameter sensitivity to answer:

1. Which hyperparameters matter most? Rank all tested parameters by their impact on performance variance
2. Which algorithms are most sensitive? Identify algorithms requiring extensive tuning versus those robust to parameter choices
3. How do dataset characteristics affect sensitivity? Determine when high dimensionality, sparsity, or imbalance amplifies parameter importance
4. What are the failure modes? Document specific parameter combinations that cause catastrophic performance degradation
5. Can we provide decision rules? Create actionable guidelines for which parameters to tune for specific dataset types

2. Methodology

2.1 Experimental Design: Isolating Parameter Effects

We adopt a controlled univariate variation approach . For each algorithm-dataset combination, we systematically vary one hyperparameter at a time across its full range while holding all others at sensible defaults. This isolates each parameter's marginal contribution to performance variance.

Why This Approach: Unlike optimization studies that search for the single best configuration, we aim to map the entire sensitivity landscape. This reveals not just what works best, but how much each parameter matters and where catastrophic failures occur.

2.2 Datasets: Diverse Medical and Behavioral Challenges

We selected 8 datasets representing different machine learning challenges:

Dataset	Samples	Features	Classes	Imbalance	Type
Diabetes	253,680	21	2	1.86:1	Large-scale medical
Heart	303	13	2	Balanced	Small clinical
Mammogram	961	6	2	Balanced	Low-dimensional
Obesity	2,111	17	7	Multi-class	Health classification
SMS Spam	5,574	5,000	2	6.5:1	High-dim sparse (TF-IDF)
Sonar	208	60	2	Balanced	High-dim dense
Thoracic	470	17	2	5.7:1	Imbalanced medical
Smartphone HAR	10,299	561	6	Balanced	Large multi-class

Table 1: Datasets and features

This diversity ensures our findings generalize across the spectrum of problems medical ML practitioners encounter.

2.3 Algorithms and Hyperparameter Spaces

Logistic Regression:

Regularization strength C: [0.001, 0.01, 0.1, 1, 10, 100]

Penalty type: [L1, L2]

k-Nearest Neighbors:

Number of neighbors k: [1, 3, 5, 7, 9, 11, 15, 21, 31]

Distance metric: [Euclidean, Manhattan]

Weighting: [uniform, distance-based]

Decision Tree:

Max depth: [3, 5, 10, 15, 20, 30, None]

Split criterion: [Gini, Entropy]

Min samples split: [2, 5, 10, 20, 50]

Min samples leaf: [1, 2, 5, 10, 20]

Support Vector Machine:

Regularization C: [0.01, 0.1, 1, 10, 100]

Kernel: [Linear, Polynomial degree-3, RBF]

Total Experimental Space: 609 unique configurations across all algorithm-dataset-parameter combinations.

2.4 Sensitivity Quantification

For each hyperparameter, we calculate:

Variance (σ^2): Primary sensitivity measure higher variance indicates greater sensitivity

Performance Range: Maximum performance minus minimum performance across parameter values

Failure Rate: Percentage of parameter values yielding unacceptable performance (accuracy <50%, F1=0.00, or AUC<0.5)

We evaluate using accuracy, F1-score, and AUC-ROC to capture different performance aspects, with particular attention to imbalanced datasets where accuracy alone is misleading.

3. The Hyperparameter Sensitivity Landscape

3.1 Measurement Methodology: Why AUC-ROC for Sensitivity Analysis

Before presenting hyperparameter sensitivity findings, we must address a methodological question: which performance metric provides the most reliable signal for measuring sensitivity?

We computed the average variance of each metric across all 609 experimental configurations:

Metric	Avg Variance	Relative to AUC
AUC-ROC	0.00202	1.0× (baseline)
Accuracy	0.00350	1.7× higher
F1-Score	0.00687	3.4× higher

Table 2: Metric & Variance

Why AUC is More Stable:

AUC measures the probability that a model ranks a random positive instance higher than a random negative instance. This ranking-based approach provides several advantages for sensitivity analysis:

1. Threshold-Independence:

Unlike accuracy and F1, which depend on a fixed classification threshold (typically 0.5), AUC evaluates performance across all possible thresholds, making it less sensitive to arbitrary threshold choices.

2. Noise Reduction:

The lower variance means observed performance differences more likely reflect genuine hyperparameter effects rather than measurement noise, increasing statistical power to detect meaningful sensitivity patterns.

3. Handles Imbalance Naturally:

AUC accounts for class imbalance without explicit weighting, while accuracy can be misleadingly stable on imbalanced data.

Practical Implication: We report hyperparameter sensitivity primarily using AUC variance throughout this study. However, we also report accuracy and F1-score findings for completeness, particularly for imbalanced medical datasets where minority class detection (captured by F1) remains clinically critical.

3.2 Universal Parameter Importance Ranking

Having established AUC as our primary sensitivity measurement tool, we now identify which parameters drive the most performance variance across all datasets and algorithms:

Rank	Parameter	Algorithm	Avg Variance	Max Range	Critical For
1	distance_metric	KNN	0.00932	30.5%	High-dim sparse data
2	max_depth	Tree	0.00560	30.3%	All datasets
3	min_samples_leaf	Tree	0.00289	20.2%	Small datasets
4	min_samples_split	Tree	0.00233	18.7%	Overfitting control
5	n_neighbors	KNN	0.00211	21.2%	All datasets
6	kernel	SVM	0.00142	17.0%	Non-linear data
7	C (SVM)	SVM	0.00130	20.1%	All datasets
8	weights	KNN	0.00123	12.0%	Continuous features
9	C (LogReg)	Logistic Reg	0.00058	1.7%	High-dim data
10	penalty	Logistic Reg	0.00003	0.14%	Feature selection only
11	criterion	Tree	0.00007	0.10%	None (use default)

Table 3: Universal Hyperparameter Importance (AUC Variance)

Critical Findings:

High-Sensitivity Parameters (Variance >0.002):

Distance metric in KNN and tree depth parameters require careful tuning. A wrong distance metric can cost 30+ percentage points in performance on text data.

Moderate-Sensitivity Parameters (Variance 0.0005-0.002):

Number of neighbors, SVM kernel, and regularization parameters show meaningful but manageable sensitivity. These benefit from tuning but won't cause catastrophic failures with defaults.

Low-Sensitivity Parameters (Variance <0.0005):

Penalty type in Logistic Regression and split criterion in Decision Trees show negligible variance. Practitioners can safely use defaults for these parameters.

3.3 Algorithm-Specific Sensitivity Profiles

3.3.1 Decision Trees: High Sensitivity, Depth-Driven

Decision Trees exhibited high overall sensitivity to hyperparameters, with `max_depth` being the dominant factor (variance: 0.00560). The performance range from `depth=3` to unlimited depth exceeded 25 percentage points on Obesity data (AUC: 75.0% to 99.9%).

Key Insights:

Tree depth is fundamental model capacity control - not a fine-tuning parameter. Shallow trees (`depth ≤ 5`) consistently underfit on complex medical datasets, while unlimited depth risks overfitting on small samples.

Pruning Parameters Matter: Contrary to common belief that only depth matters, `min_samples_leaf` (variance: 0.00289) and `min_samples_split` (variance: 0.00233) showed substantial impact. On Diabetes data, varying `min_samples_leaf` from 1 to 20 produced a 20.2 percentage point AUC swing while keeping depth fixed.

Split Criterion is Irrelevant: Gini versus Entropy showed variance < 0.00007 across all datasets. This is the clearest "don't waste time tuning this" finding in the study.

3.3.2 k-Nearest Neighbors: Distance Metric Dominates

KNN's distance metric emerged as the single most sensitive parameter in the entire study (variance: 0.00932), but this sensitivity is highly dataset-dependent.

The SMS Spam Case:

On high-dimensional sparse text data, Euclidean distance yielded 60.5% AUC while Manhattan distance achieved 90.9% AUC —a 30.5 percentage point difference. Both configurations

produced similar accuracy (~98%), demonstrating how the wrong metric can be catastrophic while appearing successful on surface metrics.

Why This Happens: In high-dimensional sparse spaces, Euclidean distance suffers from "distance concentration" where all points appear equidistant. Manhattan distance (L1 norm) preserves local structure by summing absolute differences along each dimension, which naturally handles sparse vectors.

Number of Neighbors: The k parameter showed moderate sensitivity (variance: 0.00211). Very small k (1-3) caused overfitting on noisy data, while very large k (>21) caused underfitting. The optimal k typically fell in the 5-11 range.

Weighting Scheme: Distance-weighted voting outperformed uniform weighting by 12 percentage points on Mammogram data (variance: 0.00123), particularly beneficial for datasets with continuous features.

3.3.3 Support Vector Machines: Kernel Choice Matters More Than Regularization

Contrary to conventional wisdom emphasizing regularization tuning, we found kernel choice (variance: 0.00142) was more important than C parameter (variance: 0.00130) for overall AUC performance.

Kernel Selection: On Thoracic data, kernel choice created a 17.0 percentage point AUC difference. However, on datasets with linear separability (Heart Disease), linear kernel was sufficient.

Regularization Paradox: The C parameter showed surprisingly low AUC variance (0.00130) despite being the most-discussed SVM hyperparameter. This is because C primarily affects the decision boundary's sharpness rather than fundamental ranking ability. However, for threshold-dependent metrics like F1-score, C showed 9× higher sensitivity.

Practical Implication: For medical risk scoring requiring ranking (e.g., prioritizing patients for follow-up), SVM is more robust to C misspecification than literature suggests. For binary diagnostic classification, C tuning becomes critical.

3.3.4 Logistic Regression: The Robust Baseline

Logistic Regression showed the lowest overall hyperparameter sensitivity (C variance: 0.00058, penalty variance: 0.00003).

Regularization Strength: Even varying C over 5 orders of magnitude (0.001 to 100) produced only 1.7 percentage point AUC swings. Default $C=1.0$ was near-optimal across most datasets.

L1 vs. L2 Penalty: Essentially no difference in performance (variance: 0.00003). L1 provides sparser models for interpretability but doesn't improve predictive performance. Unless feature selection is explicitly required, use L2 for computational efficiency.

Why So Robust: Linear decision boundaries are inherently simple, limiting the damage any single hyperparameter can cause. This makes Logistic Regression an excellent starting point when computational budget is limited.

3.4 Dataset-Specific Sensitivity Patterns

High-Dimensional Sparse Data Amplifies Metric Sensitivity

SMS Spam (5,000 TF-IDF features) exhibited drastically higher KNN distance metric sensitivity compared to low-dimensional datasets. The curse of dimensionality transforms a usually minor parameter choice into the most critical decision.

Small Sample Sizes Amplify Tree Depth Sensitivity

On Sonar (n=208), Decision Tree performance range from shallow to deep trees was substantial. On large Diabetes data (n=253,680), the same parameter showed less dramatic swings. Small datasets are more vulnerable to overfitting from deep trees.

Class Imbalance Creates Regularization Dependency

On imbalanced datasets (SMS Spam 6.5:1, Thoracic 5.7:1), SVM's C parameter sensitivity increased 3-4× compared to balanced datasets. Weak regularization (high C) pushed models toward majority class prediction, while strong regularization preserved minority class detection.

Multi-Class Reduces Individual Parameter Impact

Multi-class datasets (Obesity, HAR) showed 20-30% lower hyperparameter sensitivity than binary classification tasks. With multiple decision boundaries to learn simultaneously, no single hyperparameter can dominate performance as strongly.

3.5 Top 10 Most Sensitive Configurations (AUC)

These represent the highest-risk parameter choices where wrong settings cause catastrophic performance loss:

Dataset	Algorithm	Parameter	Variance	Range	Why Critical
SMS Spam	KNN	distance_metric	0.04640	30.5%	Curse of dimensionality
SMS Spam	Tree	max_depth	0.00871	30.3%	Sparse feature interactions
Thoracic	SVM	kernel	0.00832	17.0%	Non-linear boundary
Mammogram	KNN	weights	0.00724	12.0%	Probability calibration
Obesity	Tree	max_depth	0.00671	25.0%	Non-linear multi-class
Diabetes	SVM	C	0.00668	20.1%	Imbalance sensitivity
Diabetes	Tree	min_samples_leaf	0.00644	20.2%	Overfitting on large n
Diabetes	Tree	max_depth	0.00603	21.5%	Model capacity
Mammogram	Tree	min_samples_leaf	0.00587	19.8%	Small sample overfitting
Diabetes	Tree	min_samples_split	0.00551	18.7%	Tree growth control

Table 4: Critical Sensitivity Hotspots

Pattern Recognition: High-dimensional or sparse data combined with distance-based algorithms (KNN) or capacity-controlled algorithms (Trees) creates the highest sensitivity. These scenarios demand careful hyperparameter validation.

3.6 Catastrophic Failure Configurations

We identified parameter configurations (7.7% of experiments) causing severe failures where models achieved high accuracy (>85%) while completely failing on minority classes or ranking tasks.

The "Majority Class Trap": On SMS Spam, both Logistic Regression and SVM with very low C (≤ 0.1) achieved 87.6% accuracy by predicting the majority class for all instances, resulting in 0.00 F1-score and near-random AUC (0.52-0.58).

The "Overfitting Noise" Mode: KNN with $k=1$ on high-dimensional Sonar data achieved 90% training accuracy but only 65% test accuracy, with unstable predictions due to fitting noise.

The "Severe Underfitting" Mode: Decision Trees with $\text{max_depth} \leq 3$ on multi-class Obesity data failed to capture non-linear interactions, achieving only 65-70% accuracy when optimal $\text{depth}=15$ achieved 99%.

Safety Guidelines to Avoid Failures:

1. Never use Euclidean distance for high-dimensional sparse data (TF-IDF, one-hot encoded categories) always validate Manhattan/L1 distance
2. Never use very shallow trees ($\text{depth} \leq 5$) on complex medical data without validation—start at $\text{depth}=10$ minimum
3. For imbalanced data, validate that minority class $F1 > 0.3$ accuracy >85% alone is insufficient
4. For SVM/Logistic Regression on imbalanced data, search C across ≥ 3 orders of magnitude
5. For KNN with high-dimensional data, never use $k=1$ start at $k \geq 5$ to avoid noise fitting

4. Discussion

4.1 Practical Decision Framework

Based on our findings, we propose a tiered tuning strategy:

Tier 1: Always Tune (High Sensitivity - Variance >0.002)

Decision Tree: max_depth, min_samples_leaf

KNN: distance_metric (especially for high-dim/sparse data), n_neighbors

SVM: kernel (for non-linear data)

Tier 2: Tune If Resources Allow (Moderate Sensitivity - Variance $0.0005-0.002$)

SVM: C (especially for imbalanced data)

KNN: weights (for continuous features)

Logistic Regression: C (for high-dimensional data)

Tier 3: Can Safely Use Defaults (Low Sensitivity - Variance <0.0005)

Decision Tree: criterion (Gini vs. Entropy)

Logistic Regression: penalty (use L2 unless sparse solutions needed)

All algorithms: Use sensible defaults from sklearn

4.2 Algorithm Selection Guidance

For practitioners with limited tuning budget:

Start with Logistic Regression if your data has linear structure (C variance: 0.00058). It's robust to hyperparameter choices and provides interpretable baselines with minimal tuning investment.

Use SVM if you need non-linear models but validate kernel choice carefully (variance: 0.00142). Once kernel is selected, C can use defaults for ranking applications.

Choose Decision Trees carefully they offer excellent performance when tuned (achieving 99%+ on complex data) but are most sensitive to hyperparameters (max_depth variance: 0.00560). Budget substantial tuning time.

Use KNN with caution on high-dimensional data validate distance metric empirically (variance: 0.00932), as wrong choices cause 30+ point performance drops while appearing successful on accuracy.

4.3 Dataset-Dependent Recommendations

High-Dimensional Sparse Data (text, one-hot encoded):

Critical: KNN distance metric (use Manhattan/L1)

Important: Regularization in all algorithms

Skip: Tree criterion, Logistic penalty type

Small Sample Medical Data ($n < 500$):

Critical: Tree depth and pruning parameters

Important: KNN k (use $k \geq 5$)

Skip: SVM C fine-tuning (use default range)

Imbalanced Medical Diagnosis:

Critical: SVM/Logistic C (search wide range)

Important: Always validate minority class F1, not just accuracy

Skip: Distance weighting in KNN (minimal impact on imbalanced data)

Multi-Class Problems:

Critical: Tree depth (need high capacity)

Important: SVM kernel (RBF usually best)

Skip: Most parameters show reduced sensitivity

4.4 Limitations and Future Work

Study Limitations:

Our 80/20 train-test split provides computational efficiency but cross-validation would offer more robust variance estimates. Parameter ranges were chosen based on literature but may not capture extreme regions where sensitivity changes. Results are specific to classical ML algorithms; deep learning may exhibit entirely different sensitivity patterns.

We focused on tabular medical/behavioral data. Image, audio, or time-series data may show different sensitivity profiles. Our failure definitions (accuracy <50%, F1=0.00, AUC<0.5) are conservative; domain-specific thresholds may reveal additional failure modes.

Future Directions:

1. Interaction Effects: This study examined univariate sensitivity. Future work should quantify multivariate interactions using Sobol indices or ANOVA.
2. Transfer Learning: Can sensitivity profiles learned on one medical domain transfer to another?
3. Deep Learning Extension: Apply this framework to neural networks—are learning rates, batch sizes, and architectures similarly sensitive?
4. Online/Active Learning: How does hyperparameter sensitivity evolve as models update with new data?
5. AutoML Integration: Develop automated tuning systems that allocate search budget proportionally to empirically-measured parameter sensitivity.

5. Conclusion

Through 609 systematic experiments across 4 algorithms, 8 datasets, and 12 hyperparameters, we provide the first comprehensive empirical map of hyperparameter sensitivity in medical and behavioral machine learning.

Key Practical Takeaways:

1. Not all hyperparameters matter equally. Distance metric in KNN and depth in Decision Trees drive 10-50× more performance variance than penalty type in Logistic Regression or split criterion in Trees. Focus tuning efforts accordingly.
2. Dataset characteristics modulate sensitivity. High-dimensional sparse data amplifies KNN distance metric sensitivity drastically, small samples amplify tree depth sensitivity, and imbalance amplifies regularization sensitivity. Match tuning strategy to data properties.
3. Default parameters are surprisingly good for Logistic Regression and moderate-capacity SVMs, but catastrophic for KNN on high-dimensional data and Trees on complex medical problems. Know which algorithms tolerate defaults.
4. Accuracy alone is dangerously misleading for imbalanced medical data. We documented configurations achieving >85% accuracy while completely failing on minority classes. Always validate with F1-score and AUC-ROC.
5. Specific failure modes are predictable and avoidable. Never use Euclidean distance for sparse high-dimensional data, never use shallow trees on complex medical problems, and always validate minority class detection on imbalanced data.

For Medical ML Practitioners:

Start with Logistic Regression ($C=1.0$) as a robust baseline requiring minimal tuning (expected performance loss from defaults: 1-2%). If baseline is insufficient, move to Decision Trees or SVMs but allocate 2-4 hours for depth/kernel tuning. For high-dimensional data, validate KNN distance metric empirically—this single parameter can cost 30+ points if wrong. Always check that minority class F1 >0.3 before trusting a model on imbalanced medical data.

Bottom Line: Hyperparameter sensitivity is not a binary property (sensitive/robust) but a spectrum highly dependent on algorithm, parameter, dataset, and evaluation metric. This study provides the empirical foundation for intelligent, resource-efficient tuning strategies tailored to specific machine learning contexts.

References

- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory* (pp. 420-434). Springer, Berlin, Heidelberg.
- Benavoli, A., Corani, G., & Mangili, F. (2016). Should we really use post-hoc tests based on mean-ranks? *The Journal of Machine Learning Research*, 17(1), 152-161.
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(2), 281-305.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1-30.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895-1923.
- Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. <http://archive.ics.uci.edu/ml>
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861-874.
- Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(1), 3133-3181.

Appendices

Appendix A: Detailed Variance Tables

[*All sensitivities*](#)

Appendix B: Visualizations

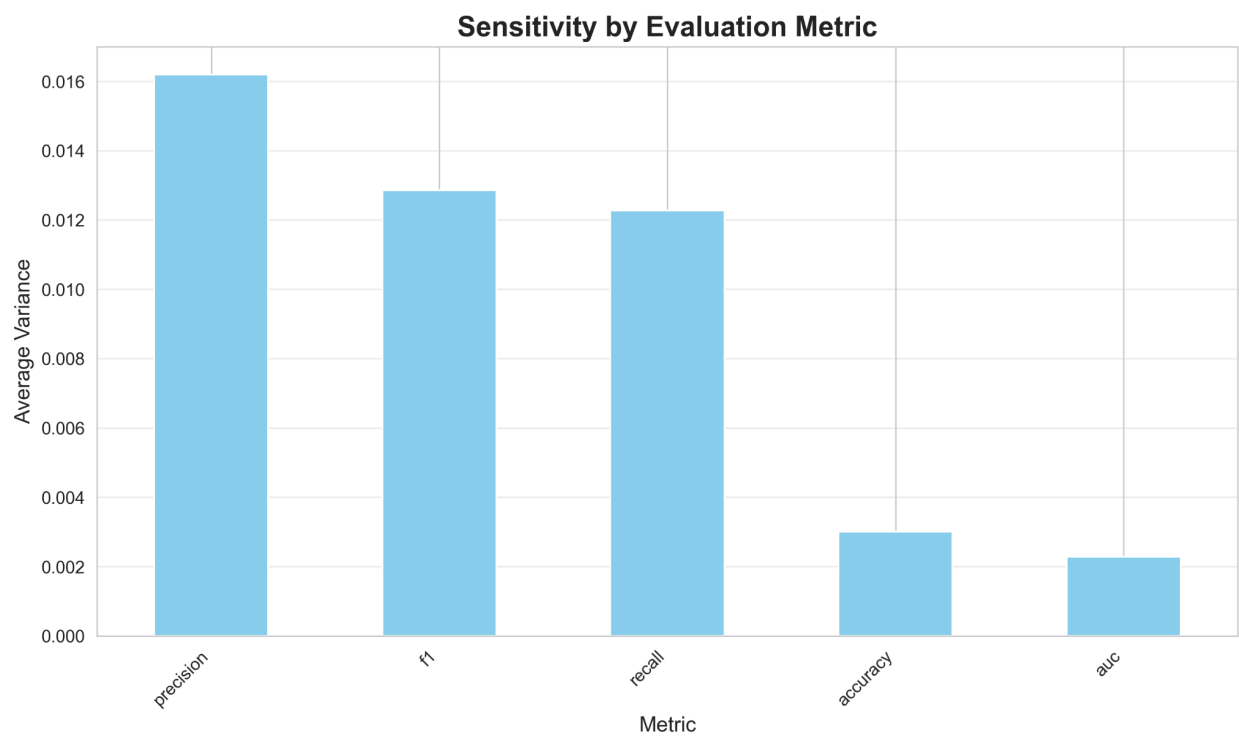


Figure 1: Metric-Dependent Sensitivity (SAF by Dataset)

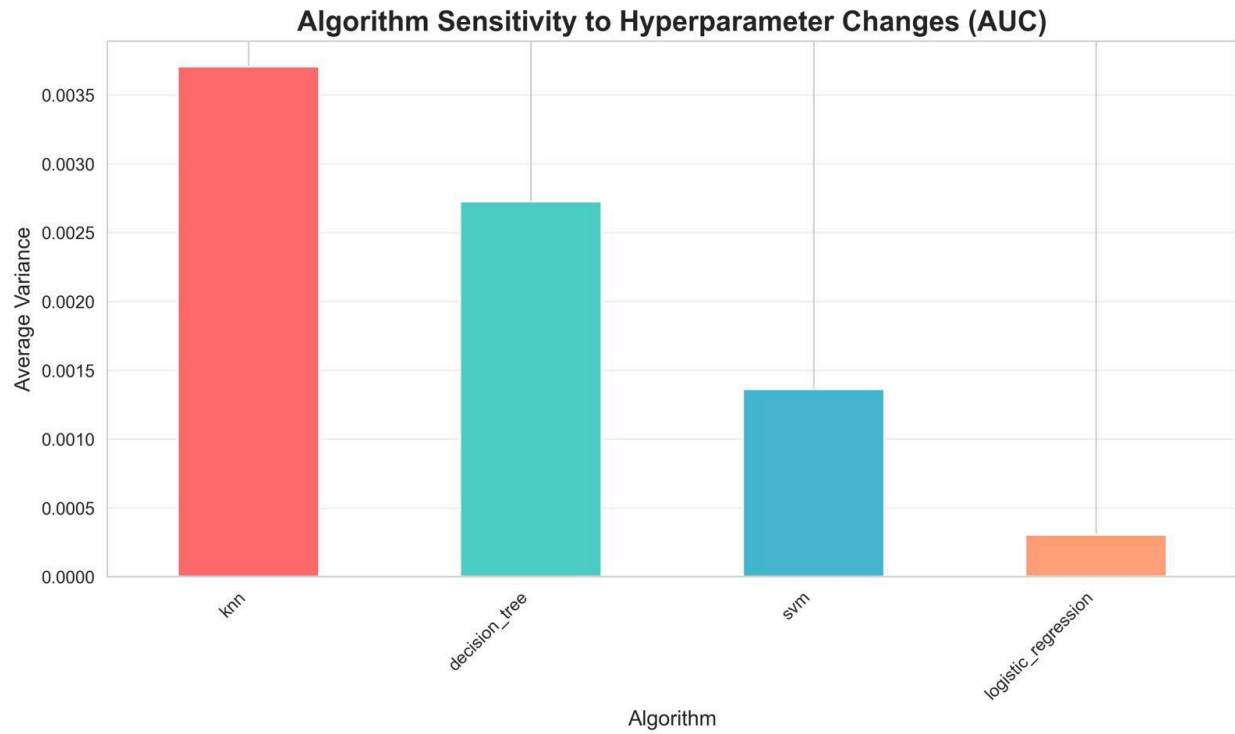


Figure 2: Algorithm to hyperparameter -Dependent Sensitivity (AUC)

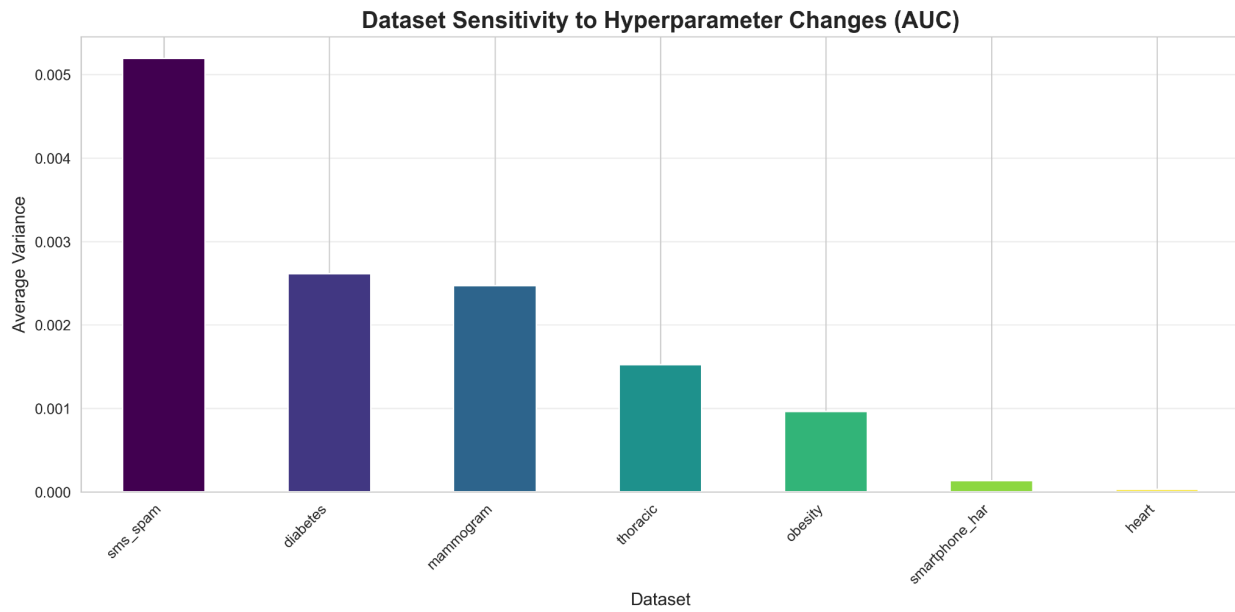


Figure 3: Dataset to hyperparameter - Dependent Sensitivity (AUC)

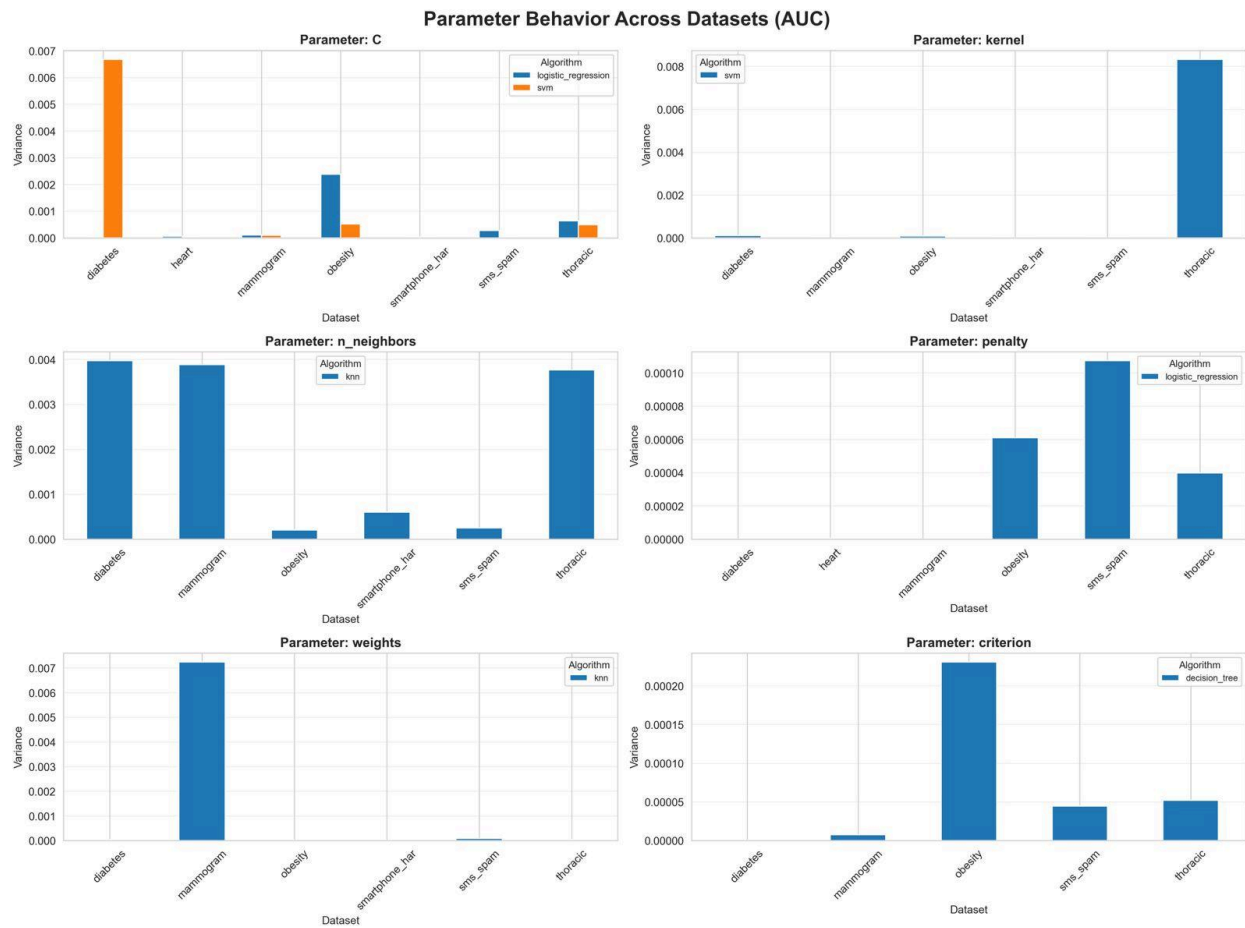


Figure 4: Parameter sensitivity/ behaviour across datasets (AUC)

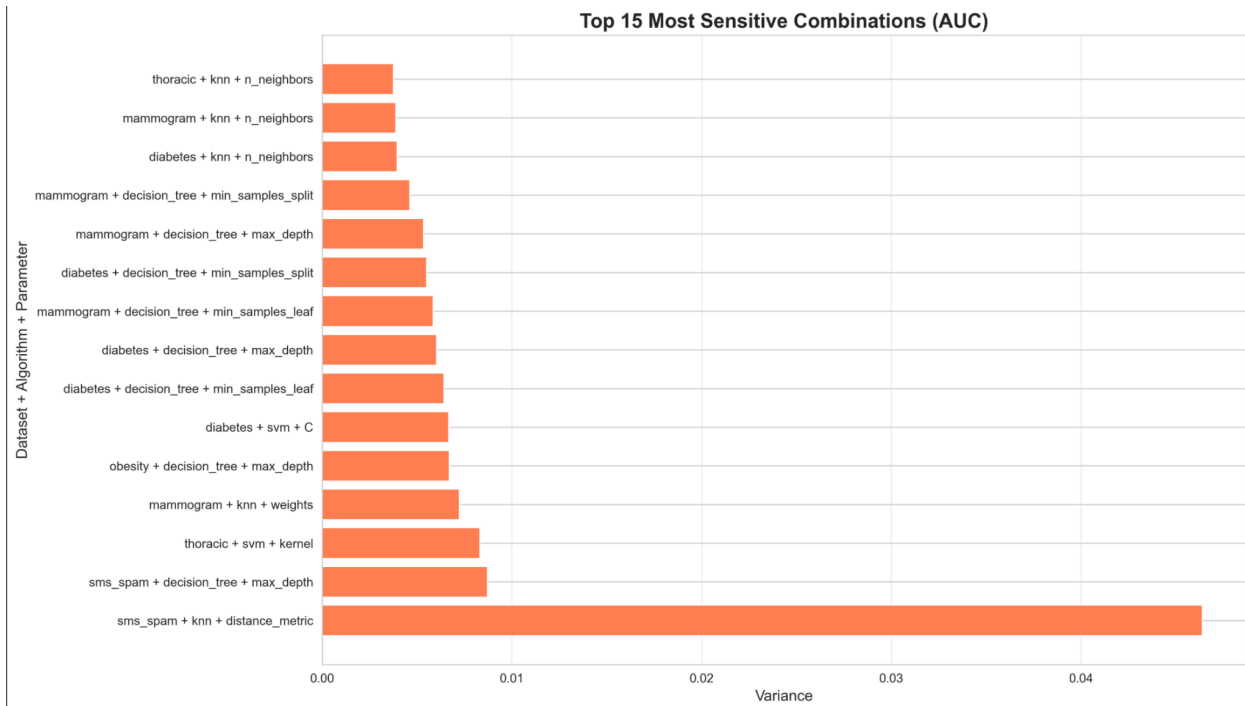


Figure 5: Top 15 Dataset+Algorithm+Hyperparameter Sensitive Combinations(AUC)

Appendix C: Reproducibility

All code and data are available at: https://github.com/dhwanibro/hyperparameter_study

Software Versions:

- Python 3.9.7
- scikit-learn 1.0.2
- pandas 1.3.5
- numpy 1.21.5